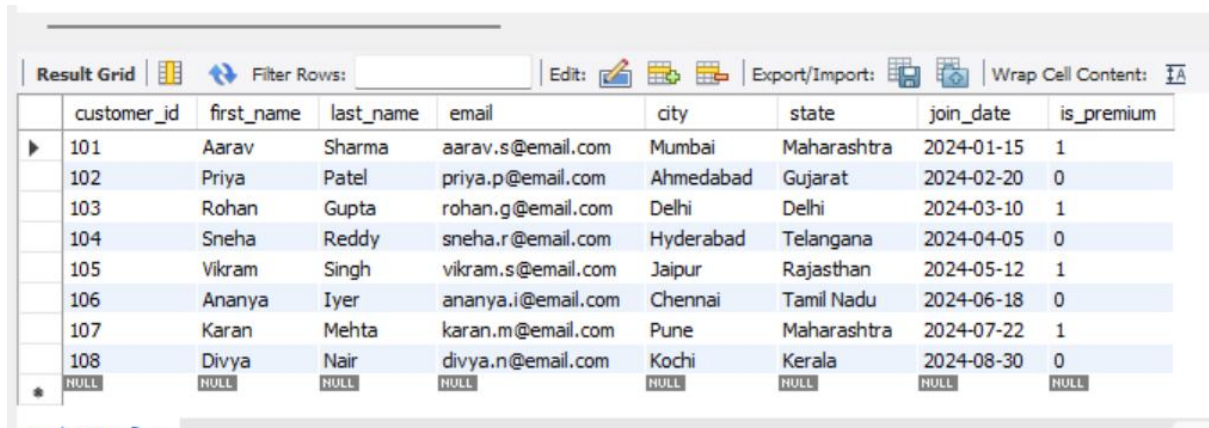


Week-2 Assignment -----SQL BASICS

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

```
SELECT * from customers;
```

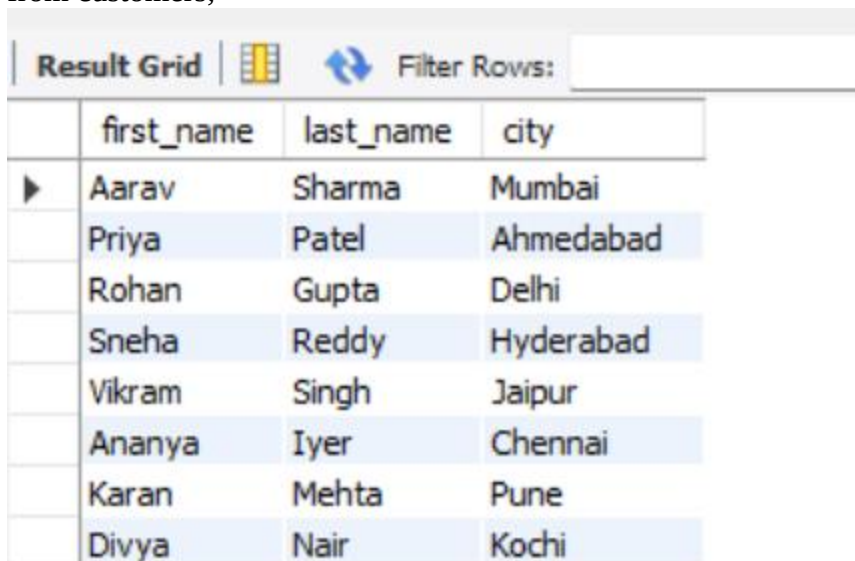


The screenshot shows a database query result grid with the following columns: customer_id, first_name, last_name, email, city, state, join_date, and is_premium. The data is as follows:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

```
SELECT first_name, last_name, city  
from customers;
```



The screenshot shows a database query result grid with the following columns: first_name, last_name, and city. The data is as follows:

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

```
SELECT DISTINCT category  
from products;
```

Result Grid	
	category
▶	Clothing
	Electronics
	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

The primary Key of all the tables are as follows:

-customers table: customer_id

-products table: product_id

-orders table: order_id

-order_items table: item_id

The primary key must be unique because:

-It prevent duplicate data.

-Eliminates data ambiguity or confusion.

-To create accurate links as foreign keys in other tables

The primary key must be not null because:

-Each row must have an identity to exist

-Database cannot reliably compare null values.

-A missing Id cannot safely connect to other tables.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

The constraints applied to the email column in the customers table are 'Unique' and 'not null'. If you tried to insert a duplicate email, the database rejects the row and throws an error of duplicate entry.

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error

```
INSERT INTO orders VALUES
(1011, 108, '2024-08-11', 'Delivered', -50.00);
```

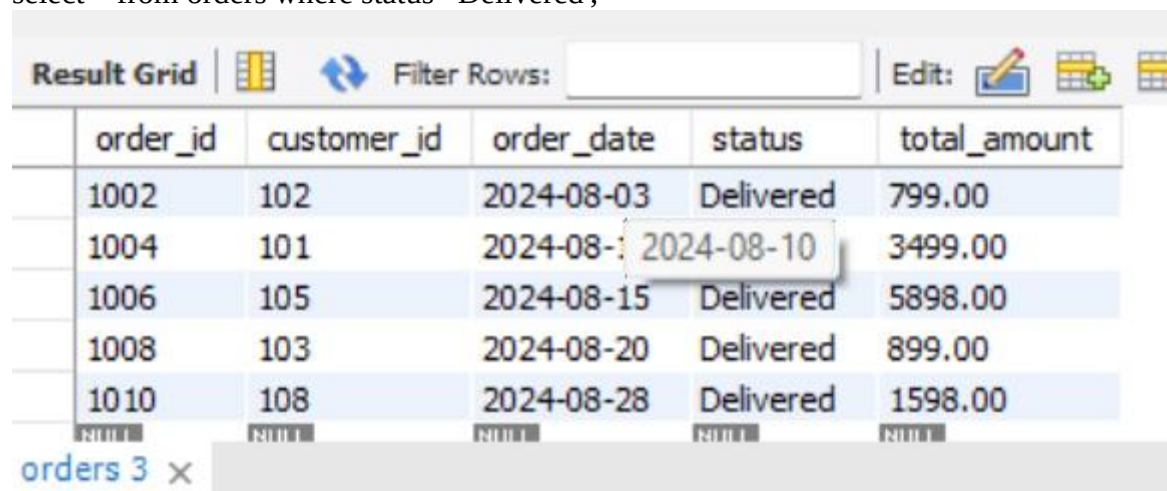
It will throw an error of check constraints as in table definition total_amount have constraint :CHECK (total_amount >= 0)

29 16:11:11 INSERT INTO orders VALUES (1011, 108, '2024-08-11', 'Delivered', -50.00) Error Code: 3819. Check constraint 'orders_chk_2' is violated. 0.047 sec

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

```
select * from orders where status='Delivered';
```



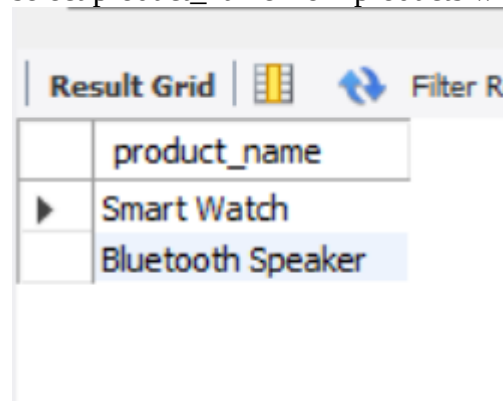
The screenshot shows a database query result grid with the following data:

order_id	customer_id	order_date	status	total_amount
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00

Below the table, it says "orders 3 x".

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

```
select product_name from products where category='Electronics' and unit_price>2000;
```



The screenshot shows a database query result grid with the following data:

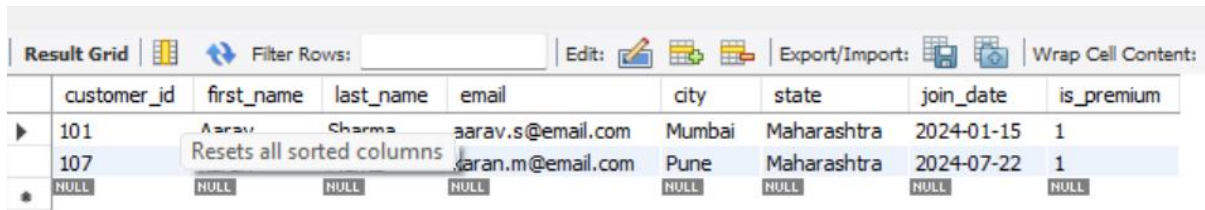
product_name
Smart Watch
Bluetooth Speaker

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

```
select * from customers
where join_date LIKE '2024%'
```

&&

state='Maharashtra';

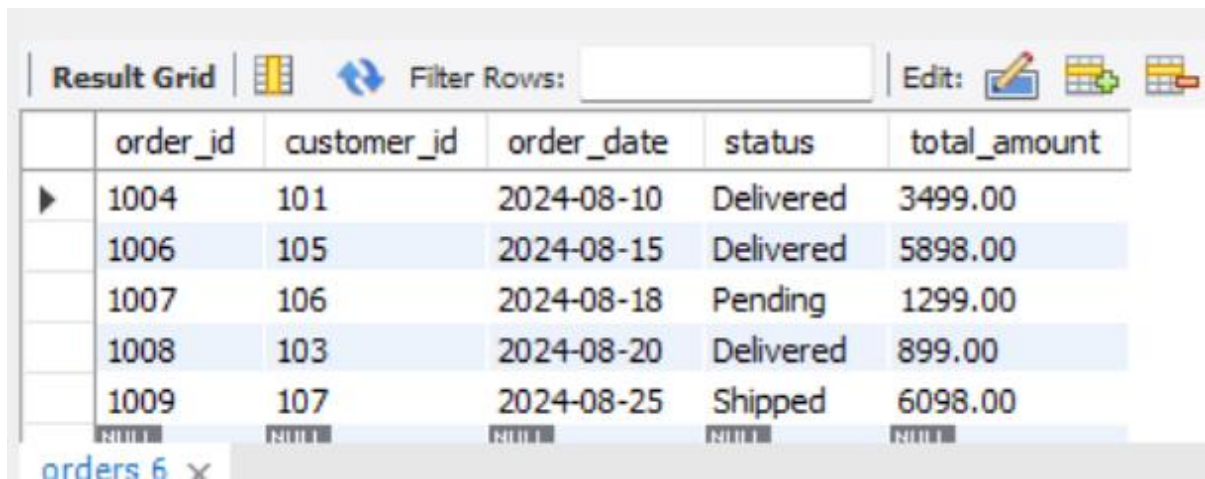


The screenshot shows a database interface with a 'Result Grid' tab. A tooltip 'Resets all sorted columns' is visible over the first_name column. The table has columns: customer_id, first_name, last_name, email, city, state, join_date, and is_premium. The data rows are:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Charma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107			aran.m@email.com	Pune	Maharashtra	2024-07-22	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

```
select * from orders
where order_date
BETWEEN '2024-08-10' AND '2024-08-25'
&&
status!='Cancelled';
```



The screenshot shows a database interface with a 'Result Grid' tab. The table has columns: order_id, customer_id, order_date, status, and total_amount. The data rows are:

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

The index idx_orders_date creates a separate, ordered lookup structure for the order_date column. Instead of scanning the entire orders table row by row, the database engine uses this index to jump directly to the requested dates. It stores the order_date values in a sorted B-Tree structure along with pointers to the actual row locations on disk.

How it improves performance:

- The database avoids reading every page of the table.
- It uses binary search matching, reducing complexity from $O(N)$ to $O(\log N)$.
- It quickly locates the start and end points for date ranges.

-Because the index is already sorted, ORDER BY order_date queries do not require an expensive sort operation in memory.

Sample query:

```
SELECT
  order_id,
  customer_id,
  total_amount
FROM
  orders
WHERE
  order_date BETWEEN '2024-01-01' AND '2024-12-31'
ORDER BY
  order_date DESC;
```

Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

No, the index on join_date will not be used because applying the YEAR() function to the column prevents the query from being SARGable (Search Argument Able).

It is because :

-Database engines cannot map the output of a function back to raw index values.

-The engine must evaluate YEAR() for every single row in the table.

-The provided schema does not actually contain an index on join_date.

Index-friendly rewrite :

```
SELECT *
FROM customers
WHERE join_date >= '2024-01-01'
  AND join_date < '2025-01-01';
```

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

```
select COUNT(*) from orders;
```

Result Grid	
	COUNT(*)
▶	10

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

```

Select SUM(total_amount)
from orders
where status ='Delivered';

```

Result Grid	
	SUM(total_amount)
▶	17191.00

Q15. Calculate the average unit_price of products in each category.

```

select category, AVG(unit_price)
from products
group by category;

```

Result Grid		
	category	AVG(unit_price)
▶	Clothing	2699.000000
	Electronics	2224.000000
	Home	949.000000

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

```

select COUNT(*) as total_orders,
SUM(total_amount) as total_revenue,
status from orders

```

group by status
order by total_revenue DESC;

Result Grid   Filter Rows:			
	total_orders	total_revenue	status
▶	6	17191.00	Delivered
	2	13596.00	Shipped
	1	2999.00	Cancelled
	1	1299.00	Pending

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

select MAX(unit_price) as Expensive,
MIN(unit_price) as Cheapest,
category from products
group by category;

Result Grid   Filter Rows:			
	Expensive	Cheapest	category
▶	4599.00	799.00	Clothing
	3499.00	899.00	Electronics
	1299.00	599.00	Home

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

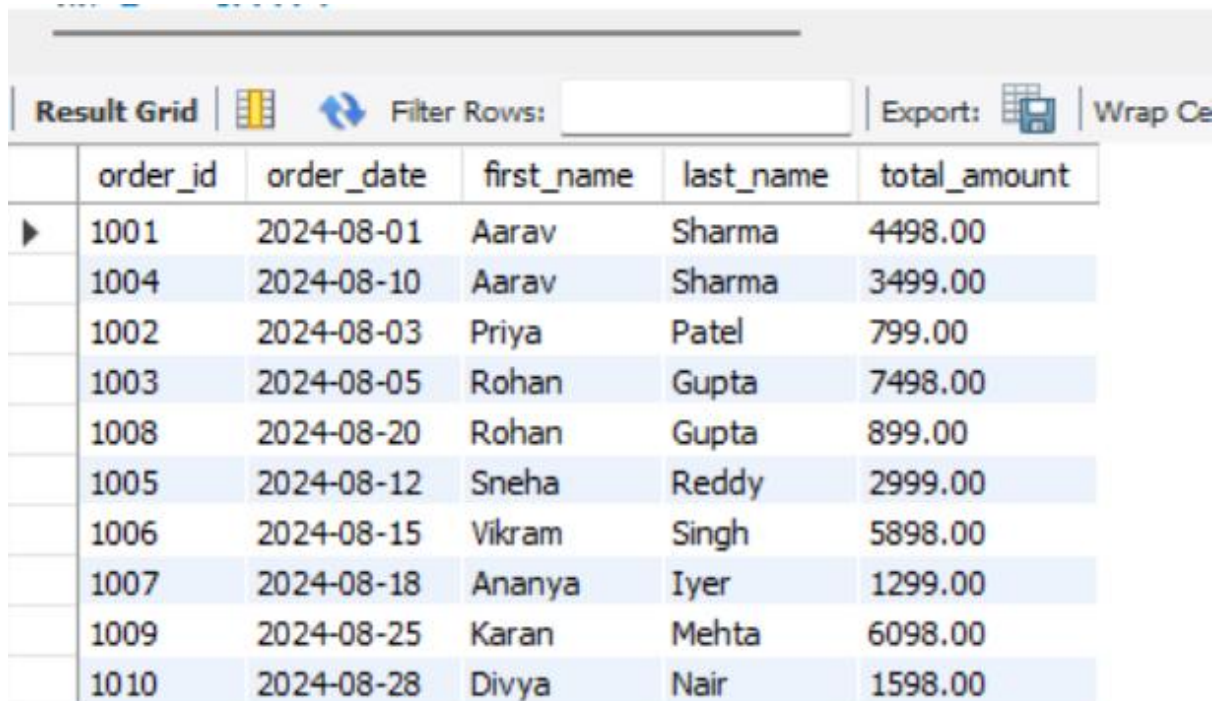
select category, AVG(unit_price) as Average_unit_price from products group by category
HAVING Average_unit_price>2000;

Result Grid   Filter Rows:		
	category	Average_unit_price
▶	Clothing	2699.000000
	Electronics	2224.000000

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

```
SELECT
    orders.order_id,
    orders.order_date,
    customers.first_name,
    customers.last_name,
    orders.total_amount
FROM orders
INNER JOIN customers
    ON orders.customer_id = customers.customer_id;
```



The screenshot shows a database interface with a 'Result Grid' tab. The grid displays the results of an SQL query. The columns are: order_id, order_date, first_name, last_name, and total_amount. The data is as follows:

	order_id	order_date	first_name	last_name	total_amount
▶	1001	2024-08-01	Aarav	Sharma	4498.00
	1004	2024-08-10	Aarav	Sharma	3499.00
	1002	2024-08-03	Priya	Patel	799.00
	1003	2024-08-05	Rohan	Gupta	7498.00
	1008	2024-08-20	Rohan	Gupta	899.00
	1005	2024-08-12	Sneha	Reddy	2999.00
	1006	2024-08-15	Vikram	Singh	5898.00
	1007	2024-08-18	Ananya	Iyer	1299.00
	1009	2024-08-25	Karan	Mehta	6098.00
	1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

```
select customers.customer_id, customers.first_name, customers.last_name, orders.order_id ,
orders.order_date, orders.total_amount from customers LEFT JOIN orders ON
customers.customer_id=orders.customer_id;
```


Result Grid						
		Filter Rows:			Export:	Wrap Cell Content:
	customer_id	first_name	last_name	order_id	order_date	total_amount
▶	101	Aarav	Sharma	1001	2024-08-01	4498.00
	101	Aarav	Sharma	1004	2024-08-10	3499.00
	102	Priya	Patel	1002	2024-08-03	799.00
	103	Rohan	Gupta	1003	2024-08-05	7498.00
	103	Rohan	Gupta	1008	2024-08-20	899.00
	104	Sneha	Reddy	1005	2024-08-12	2999.00
	105	Vikram	Singh	1006	2024-08-15	5898.00
	106	Ananya	Iyer	1007	2024-08-18	1299.00
	107	Karan	Mehta	1009	2024-08-25	6098.00
	108	Divya	Nair	1010	2024-08-28	1598.00

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

```

SELECT
    o.order_id,
    p.product_name,
    oi.quantity,
    oi.unit_price,
    oi.discount_pct
FROM
    orders o
JOIN
    order_items oi ON o.order_id = oi.order_id
JOIN
    products p ON oi.product_id = p.product_id;

```

Result Grid   Filter Rows: Export:  Wrap C					
	order_id	product_name	quantity	unit_price	discount_pct
▶	1001	Wireless Earbuds	2	1499.00	0.00
	1001	Laptop Stand	1	899.00	10.00
	1002	Cotton T-Shirt	1	799.00	0.00
	1003	Smart Watch	Smart Watch	999.00	0.00
	1003	Running Shoes	1	4599.00	5.00
	1004	Bluetooth Speaker	1	3499.00	0.00
	1005	Smart Watch	1	2999.00	0.00
	1006	Wireless Earbuds	1	1499.00	10.00
	1006	Running Shoes	1	4599.00	5.00
	1007	Bedsheet Set	1	1299.00	0.00
	1008	Laptop Stand	1	899.00	0.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

The difference between LEFT JOIN and RIGHT JOIN lies entirely in which table acts as the primary reference point for keeping unmatched rows:

-LEFT JOIN: Keeps all rows from the left table. It matches rows from the right table. Unmatched right rows return NULL.

-RIGHT JOIN: Keeps all rows from the right table. It matches rows from the left table. Unmatched left rows return NULL.

Ex:

Consider the relationship between products (left) and order_items (right).

```
SELECT p.product_name, oi.order_id
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id;
```

If a product has never been ordered, it still appears in the list with a NULL for order_id.

```
SELECT p.product_name, oi.order_id
FROM products p
RIGHT JOIN order_items oi ON p.product_id = oi.product_id;
```

If a product row was somehow deleted from the database but the transaction history remained, the order_id would still show up with a NULL for product_name.

You use a FULL Outer Join if you want to combine records from both the table regardless of whether they match or not. All the unmatched records contain null values.

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

customers.customer_id ◀—FK— orders.customer_id

orders.order_id ◀—FK— order_items.order_id

products.product_id ◀—FK— order_items.product_id

If we try to insert an order with customer_id = 999, the database will completely block the insert and throw a foreign key constraint violation error.

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

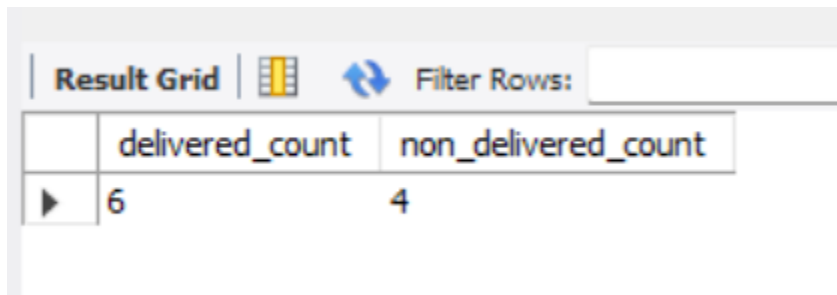
```
select product_name, unit_price,  
CASE  
  when unit_price < 1000 then 'Budget'  
  when unit_price BETWEEN 1000 AND 3000 then 'Mid-Range'  
  when unit_price > 3000 then 'Premium'  
END as price_tier  
from products;
```

Result Grid			
Filter Rows:			
	product_name	unit_price	price_tier
▶	Wireless Earbuds	1499.00	Mid-Range
	Cotton T-Shirt	799.00	Budget
	Smart Watch	2999.00	Mid-Range
	Running Shoes	4599.00	Premium
	Bluetooth Speaker	3499.00	Premium
	Bedsheet Set	1299.00	Mid-Range

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

select

```
COUNT(CASE when status='Delivered' then 1 END) as delivered_count,  
COUNT(CASE when status!='Delivered' OR status IS NULL then 1 END) as  
non_delivered_count  
from orders;
```



The screenshot shows a database query result grid. At the top, there is a tab labeled 'Result Grid' and a 'Filter Rows:' input field. Below this, the query results are displayed in a table with two columns: 'delivered_count' and 'non_delivered_count'. The first row shows the values 6 and 4 respectively.

	delivered_count	non_delivered_count
▶	6	4

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation
- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

-Atomicity: A transaction is treated as a single, indivisible "all or nothing" unit. Either every operation within the transaction successfully completes, or none of them are applied.

Ex: A fund transfer of ₹10,000 from Account A to Account B involves two operations: debiting Account A and crediting Account B. If the database crashes after the debit but before the credit, the entire transaction is rolled back. Account A is restored to its original balance, ensuring no currency is lost.

-Consistency: Ensures that a transaction can only transition the database from one valid state to another, strictly maintaining all predefined schema rules, constraints, and data integrity checks.

Ex: A banking database enforces a minimum balance constraint of ₹1,000 on savings accounts. If a user with a balance of ₹1,500 attempts a withdrawal of ₹1,000, the DBMS will abort the transaction to prevent the balance from violating the constraint by dropping to ₹500.

-Isolation: Guarantees that concurrently executing transactions are isolated from one another. The execution of one transaction must not interfere with or expose uncommitted data to another concurrent transaction.

Ex: A joint bank account holds a balance of ₹25,000. Two account holders simultaneously attempt to withdraw ₹20,000 from different ATMs. Isolation ensures the system serializes the requests; the first transaction succeeds and updates the balance to ₹5,000, causing the second transaction to fail due to insufficient funds.

-Durability: Assures that once a transaction has successfully committed, its changes are permanently recorded in non-volatile storage

Ex: A customer deposits ₹50,000 at a cash deposit machine, and the system confirms a successful transaction. If the banking server immediately suffers an ungraceful hardware failure or power loss, the recorded ₹50,000 deposit remains safely recovered on disk when the system reboots.

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
2. Insert two order items for that order
3. Update the stock_qty of the purchased products
4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.

Write the complete BEGIN...COMMIT/ROLLBACK block.

DELIMITER //

CREATE PROCEDURE ProcessOrder1011()

BEGIN

DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

ROLLBACK;

SELECT 'Transaction failed and was rolled back.' AS Status;

END;

START TRANSACTION;

INSERT INTO orders (order_id, customer_id, order_date, status, total_amount)
VALUES (1011, 102, CURDATE(), 'Pending', 1598.00);

INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES (1011, 501, 2, 499.00),
(1011, 502, 1, 600.00);

UPDATE products
SET stock_qty = stock_qty - 2
WHERE product_id = 501;

UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 502;

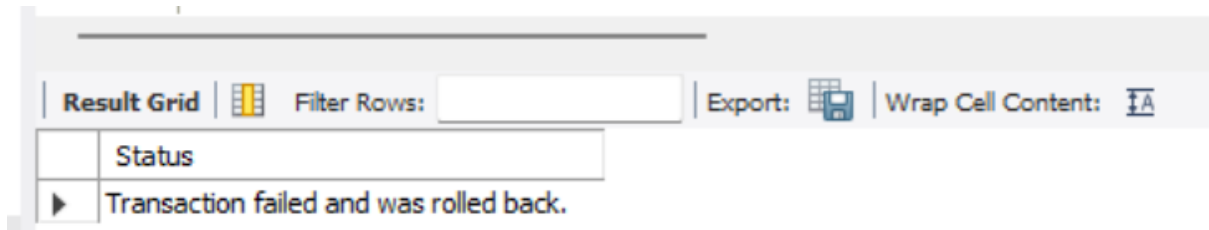
COMMIT;
SELECT 'Transaction committed successfully.' AS Status;

END //

DELIMITER ;

CALL ProcessOrder1011();

DROP PROCEDURE IF EXISTS ProcessOrder1011;



The screenshot shows a SQL client interface with a toolbar at the top. The toolbar includes a 'Result Grid' button, a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' button. Below the toolbar is a table with one row. The first column is a small square icon, and the second column contains the text 'Transaction failed and was rolled back.'

	Status
▶	Transaction failed and was rolled back.